

Architecture

What the pieces are, how they are wired together, and what happens between them at every step of a grant.

Network	Base Sepolia (chain 84532)
Core protocols	UMA Optimistic Oracle V3 · World ID · Chainlink CRE · Circle USDC
Companion docs	Project structure · CRE settlement workflow
Updated	September 2026

1. What GrantLane is

GrantLane is a milestone grant escrow where a claim has to survive a challenge before it pays.

A grantor locks USDC against a list of milestones up front. When a milestone is done, the grantee publishes their evidence and claims it with a small USDC bond. The claim is open to dispute for a fixed window. If nobody disputes it, it pays out. If someone does, UMA decides, and whoever was wrong loses their bond.

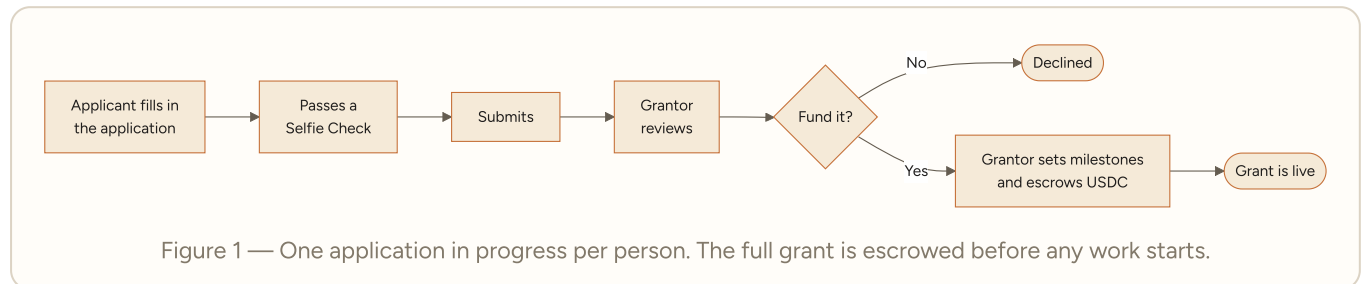
No one approves a milestone by hand, and no key can release the escrowed money. The design does not try to judge whether work is good — nothing automated can. It makes lying expensive instead: the evidence is public, the claim is bonded, and anyone can challenge it.

The idea in one line: escrow the money, then let a bonded, publicly disputable claim release it.

2. User flow

The journey from an application to a completed grant, as the people involved experience it. It has two halves: getting funded happens once per grant, and getting paid repeats for every milestone.

2.1 Getting funded



Who does what

Role	What they do
Applicant / grantee	Applies, then claims each milestone with evidence and a bond, and withdraws what is approved. A Selfie Check gates applying, claiming and changing the payout wallet.
Grantor	Reviews applications, sets the milestone scope and escrows the grant. Usually the one who disputes a weak claim.
Disputer	Anyone who believes a claim is false. They match the bond; if they are right, they take the grantee's bond.
Neutral answerer	Testnet only: a wallet with no stake in the dispute stands in for UMA's voters. The app refuses the answer to both sides.
Settler	Anyone — the grantee, a bystander, or the Chainlink CRE workflow. Settling applies an outcome; it never decides one.

2.2 Getting paid, milestone by milestone

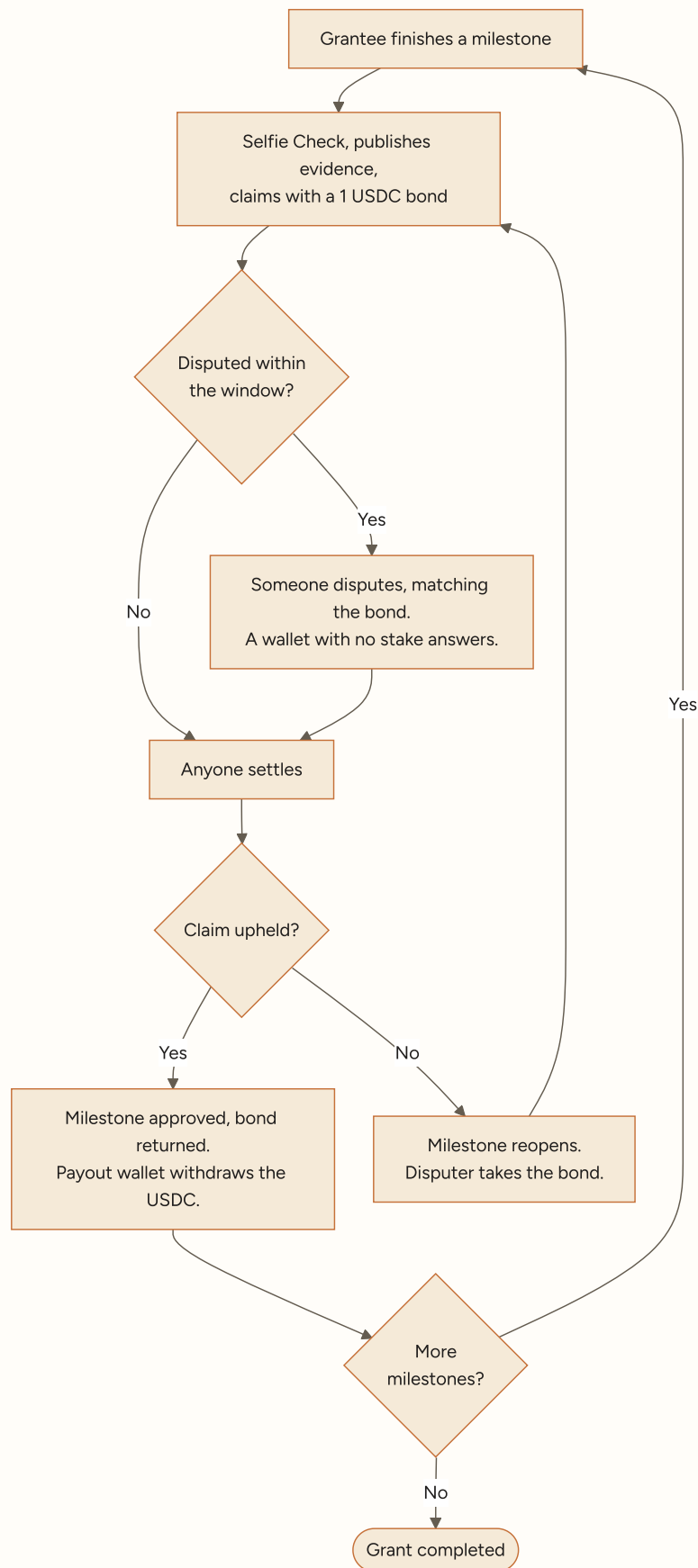


Figure 2 — An undisputed claim is always upheld. On Base mainnet the answer to a dispute comes from UMA's tokenholder vote.

3. The pieces

GrantLane is a web app in front of two smart contracts, with three external protocols doing the work the app should not do itself: proving humanness, settling disputes and running upkeep.

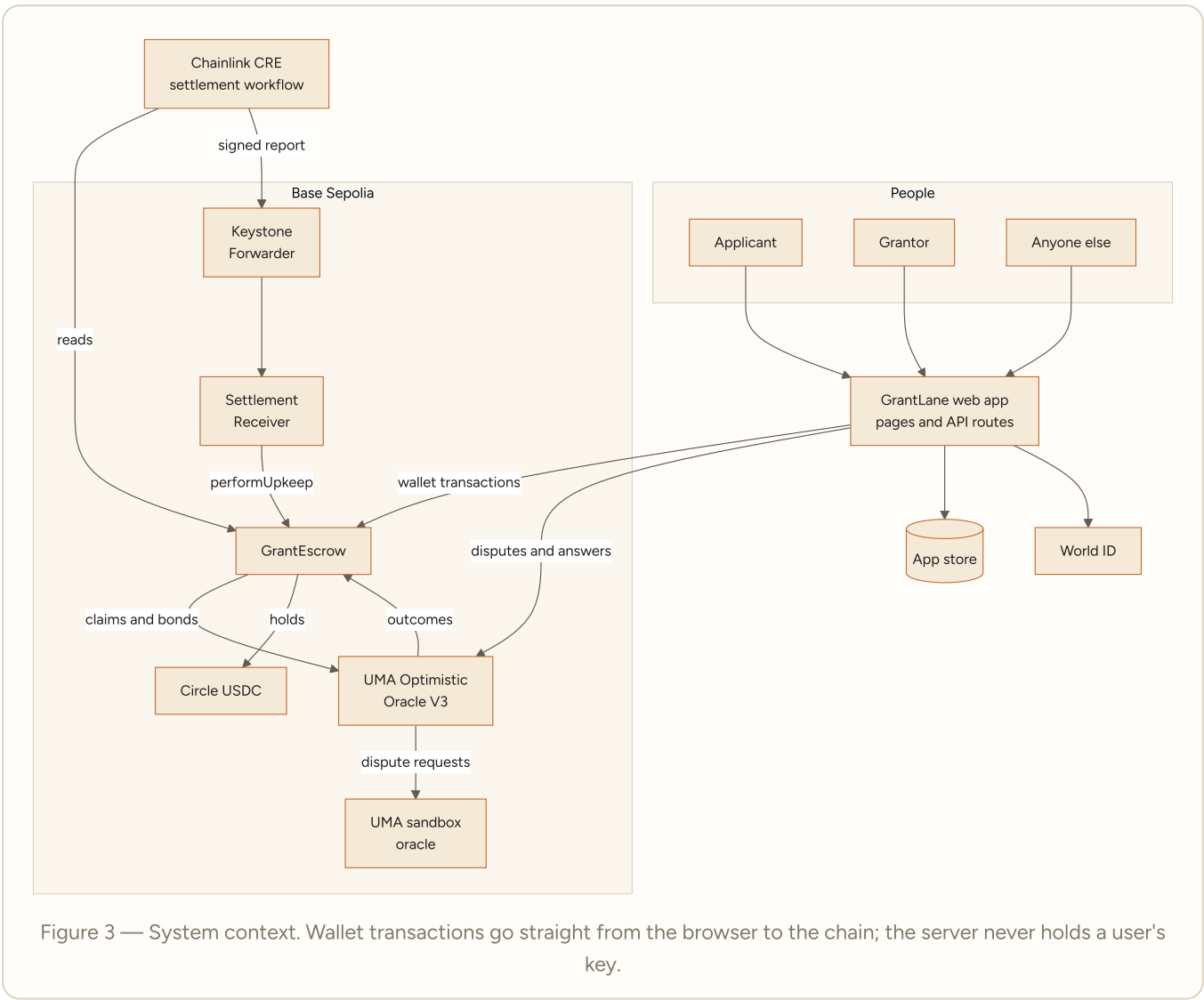


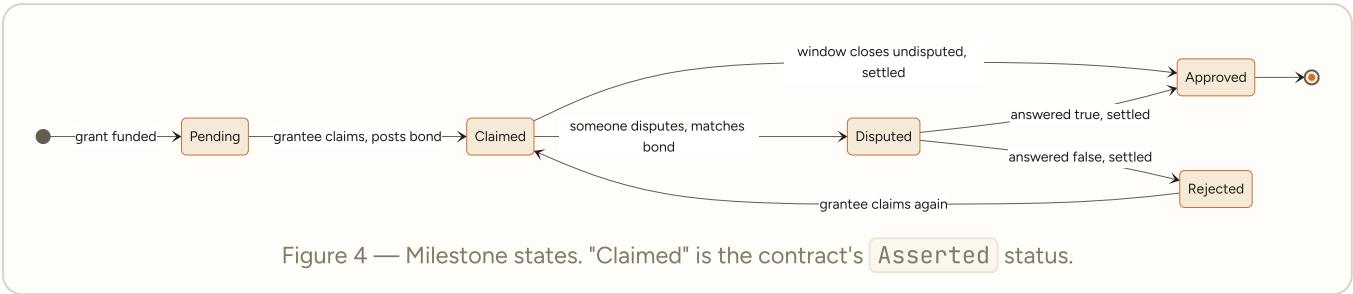
Figure 3 — System context. Wallet transactions go straight from the browser to the chain; the server never holds a user's key.

Piece	What it is	What it talks to
Web app	Next.js 14 app: applicant, grantor and evidence pages, plus API routes for the work that must run on a server.	The user's wallet (wagmi / viem), the contracts, World ID, the app store.
App store	Key-value storage: Redis in production, a JSON file locally. Holds applications and evidence bundles.	API routes only.
World ID	Selfie Check proves a live, unique human without revealing who. Proofs are verified server-side.	World App on the user's phone; World's Developer Portal verify API.

GrantEscrow	The escrow contract. Holds the USDC, turns claims into UMA assertions, credits payouts, enforces every money rule.	UMA Optimistic Oracle V3, USDC, SettlementReceiver.
UMA Optimistic Oracle V3	Holds each claim as a bonded assertion with a dispute window, and calls back into the escrow with the outcome.	GrantEscrow, the sandbox oracle.
UMA sandbox oracle	Testnet stand-in for UMA's token-holder vote: answers disputes. Replaced by the real vote on mainnet.	UMA Optimistic Oracle V3.
Circle USDC	The single currency: grant escrow, claim bond and dispute bond.	GrantEscrow, UMA.
Chainlink CRE workflow	A cron workflow that finds claims whose window has closed and settles them.	Reads GrantEscrow; writes through the forwarder and SettlementReceiver.
SettlementReceiver	Accepts the workflow's signed report and passes it to <code>GrantEscrow.performUpkeep</code> — and can call nothing else.	KeystoneForwarder, GrantEscrow.

4. A milestone's life

Every milestone moves through five states, all stored in GrantEscrow. The contract only lets it move along these arrows.



State	Meaning	Money
Pending	Funded, not yet claimed.	Held in escrow.
Claimed	Asserted on UMA; open to dispute until the window closes.	Grantee's bond held by UMA.
Disputed	Challenged; waiting for an answer.	Both bonds held by UMA.
Approved	Settled true.	Amount credited to the payout wallet; bond returned (plus the disputer's, if any, less UMA's burn).
Rejected	Settled false; can be claimed again.	Grantee's bond goes to the disputer, less UMA's burn.

A grant is **Completed** once every milestone is Approved. The grantor can close a grant and reclaim what was never released, but not while any claim is Claimed or Disputed.

5. How the pieces interact

5.1 Apply, review and fund

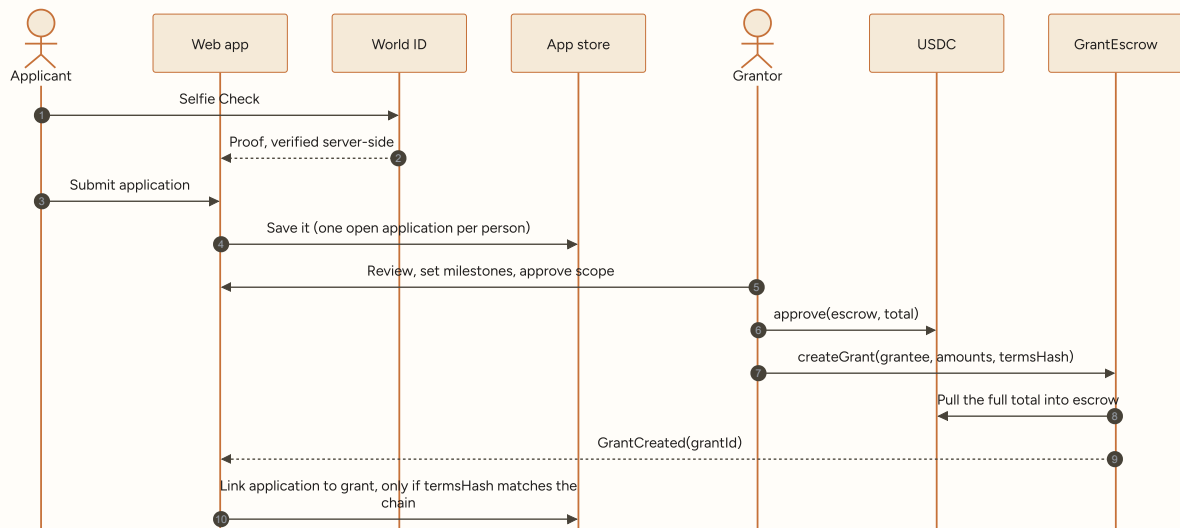


Figure 5 — The grant is escrowed in full before any work starts. The terms hash pins the milestone wording.

5.2 Claim, settle and withdraw (undisputed)

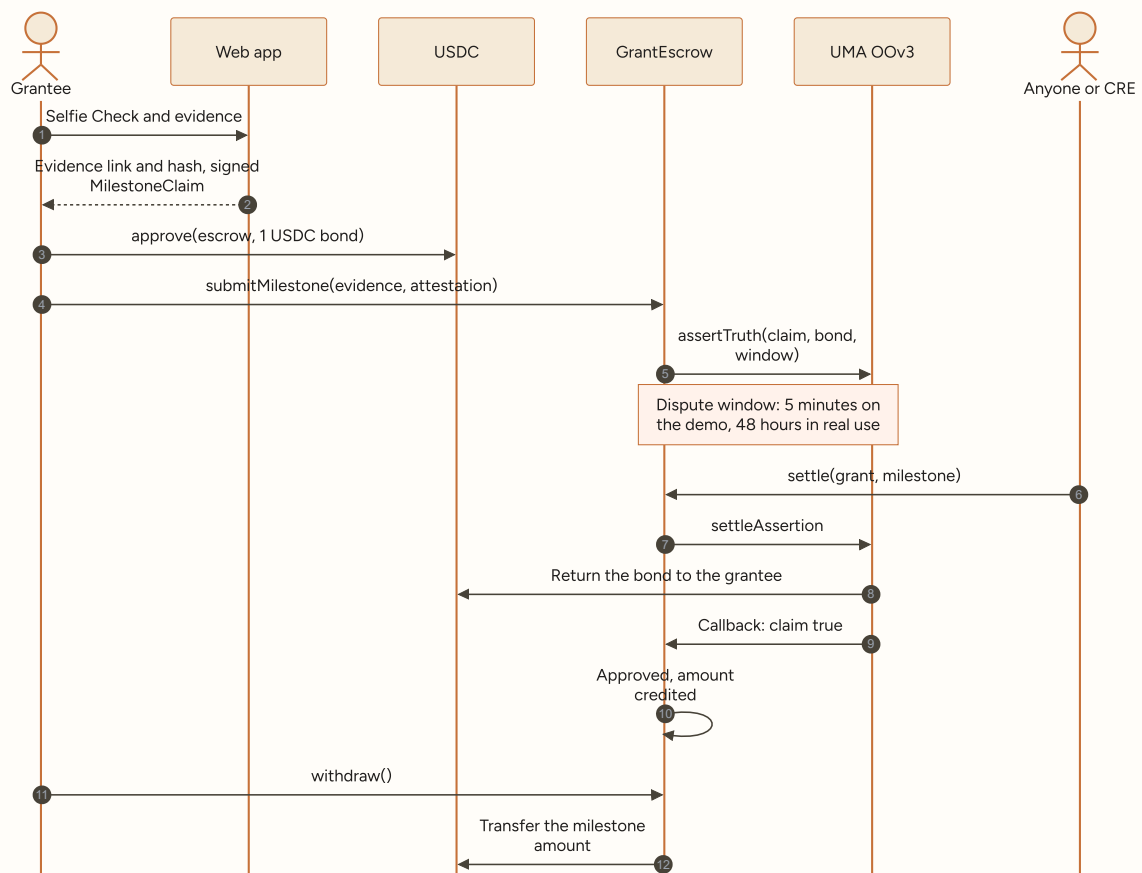


Figure 6 — Payouts are credited and withdrawn, so no failed transfer can block a settlement.

5.3 Dispute, answer and settle

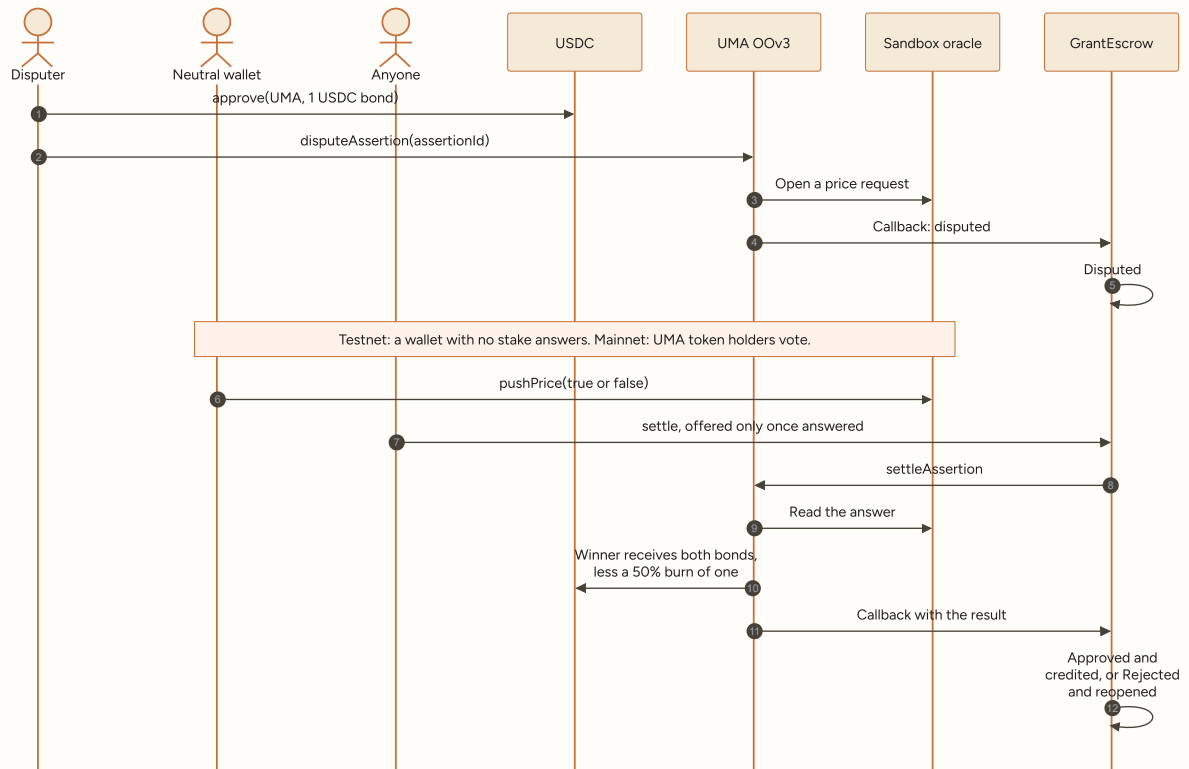


Figure 7 — Neither side of a dispute is offered the answer; settling is refused until an answer exists.

5.4 Automatic settlement with Chainlink CRE

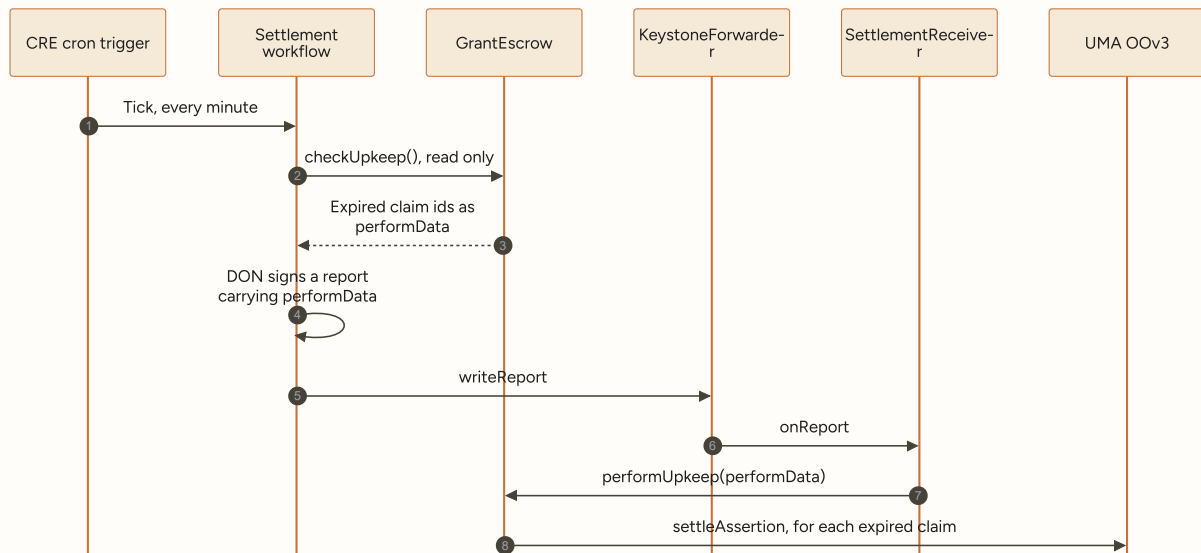
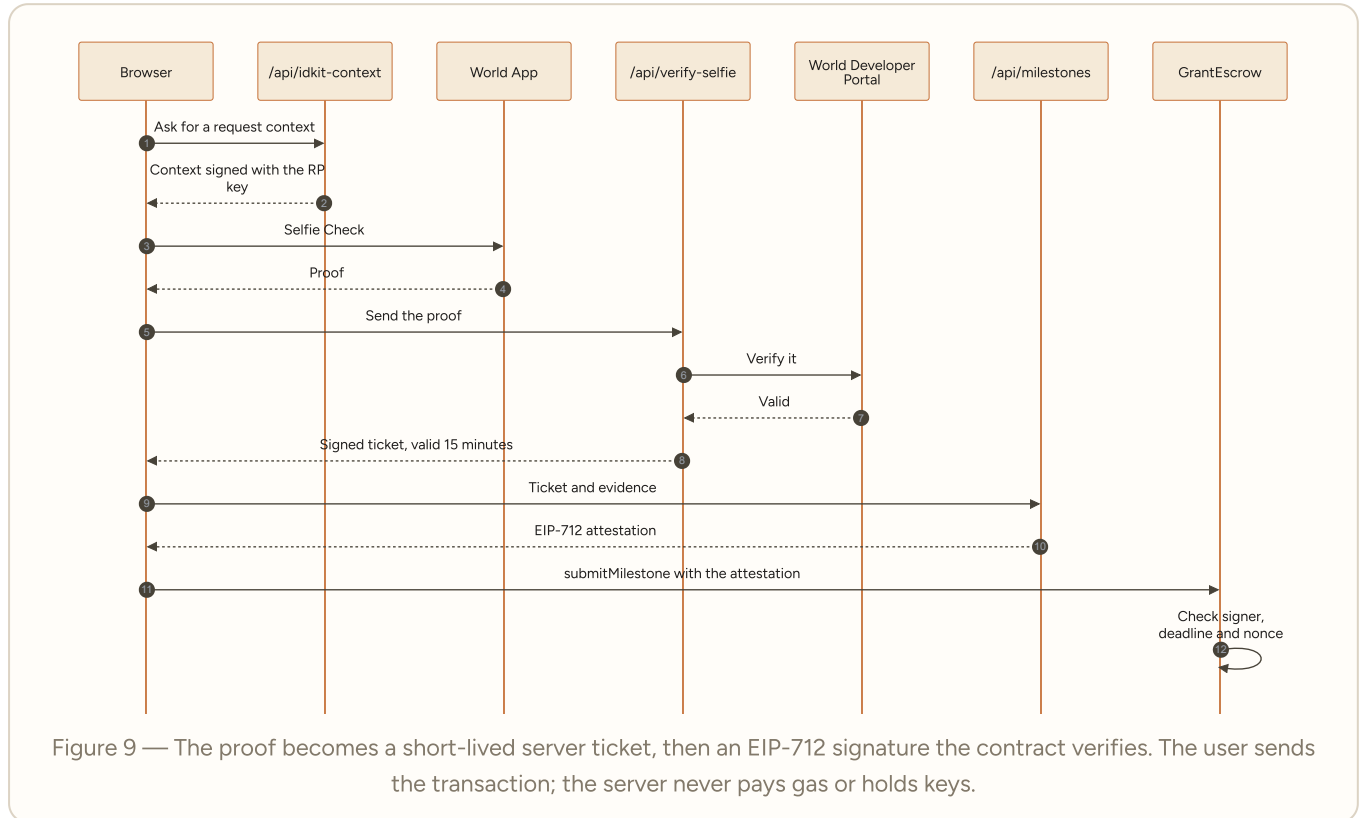


Figure 8 — The workflow is a convenience, not a gate: settle() stays open to anyone. Full detail in the CRE workflow document.

6. Proving a live human

World ID Selfie Check gates three moments: applying, claiming a milestone and changing the payout wallet. The last two end in a transaction, so the proof has to become something the contract can check. For a milestone claim it goes like this:



- **Each gate uses its own World ID action**, so a proof obtained at one gate is worthless at another.
- **Tickets** are HMAC-signed by the server, so an endpoint can tell a verified human from an invented identifier.
- **Attestations** are EIP-712 signatures from the attester key. GrantEscrow checks the signer, the deadline and a per-grant nonce, so a signature can't be replayed.
- **One open application per person**: the World ID nullifier is stable per person, so switching wallets doesn't get around it.

7. Where data lives

The chain holds everything that decides money. The app store holds what people read, and hashes on-chain prove that it hasn't been changed.

Where	What	Why there
GrantEscrow	Grant funder, grantee, payout wallet, totals, open claims, terms hash; each milestone's amount, status, dispute-window end, UMA assertion id, evidence hash; credited balances.	These decide who is paid and when, so the contract is the authority.
UMA	Each claim's text, asserter, bond, expiry, disputer and resolution.	UMA owns dispute resolution; the escrow reads outcomes through callbacks.
App store	Applications (pitch, repo snapshot, milestone titles and criteria), evidence bundles keyed by their hash.	Prose is too costly to store on-chain; the hashes make it tamper-evident.
The user's browser	Application form drafts, the short-lived verification ticket.	Convenience only; never sent anywhere until submitted.

Two integrity checks tie the stores together. The server links an application to a grant only when its milestone set hashes to the grant's on-chain `termsHash`. The evidence page recomputes both the evidence hash and the terms hash and shows whether they match the chain, so nobody has to trust the site.

8. What enforces what

Rules the contract enforces

- **Only UMA resolves claims.** The callbacks reject any other caller and ignore assertions the escrow didn't make.
- **Payouts are pulled, not pushed.** Approved amounts are credited and withdrawn, so a failed transfer can never block settlement.
- **Settlement is permissionless.** `settle` and `performUpkeep` work for anyone; `performUpkeep` re-checks every claim it is handed.
- **No closing under a live claim.** `closeGrant` is refused while any milestone is Claimed or Disputed.
- **Terms are pinned.** `createGrant` commits the terms hash, and every claim quotes it.
- **Claims need a live human.** `submitMilestone` requires a valid attestation bound to the evidence hash and the grant's claim nonce.

What each key can and cannot do

Key	Can	Cannot
Grantor wallet	Fund a grant; dispute a claim with a bond; close a grant with no open claims and reclaim unreleased funds.	Release a milestone; close under a live claim; change the terms after funding.
Grantee wallet	Claim milestones (with an attestation and bond); withdraw credited payouts; change the payout wallet (with an attestation).	Be paid without surviving the window or winning the dispute.
Attestor key (server)	Sign Selfie Check attestations for claims and payout-wallet changes.	Move money, or skip the bond or the dispute window.
Contract owner	Replace the attestor address.	Release, redirect or freeze escrowed funds.
CRE workflow	Settle claims whose window has closed.	Decide any outcome, or do anything a bystander couldn't.

9. Deployment

Live on Base Sepolia

Contract	Address
GrantEscrow	0x85AC2a3e1EBc0959599025eB6eF36eD34c862840
SettlementReceiver	0x5c7f09FDf7bC860AF7F0C5EcBD220B3391583a95
UMA Optimistic Oracle V3	0x0F7fC5E6482f096380db6158f978167b57388deE
UMA sandbox oracle	0x54e38A62ED3dC88e2B80cBA50deB940580511D26
Circle USDC	0x036CbD53842c5426634e7929541eC2318f3dCF7e
CRE KeystoneForwarder	0xF8344CFd5c43616a4366C34E3EEE75af79a74482

Escrow parameters: **1 USDC** bond for claimant and disputer alike; **5-minute** dispute window for the demo.

Testnet today, mainnet later

	Base Sepolia (now)	Base mainnet (target)
Dispute answers	UMA's sandbox oracle, answered by a wallet with no stake	UMA's token-holder vote
Dispute window	5 minutes	48 hours
Automatic settlement	Workflow built and tested; runs from the CRE CLI until deploy access is granted	Deployed to a Chainlink DON, settling every minute
Selfie Check	World ID Selfie Check (the <code>selfieCheckLegacy</code> preset)	

Configuration

Every setting the project reads is listed and explained in `.env.example` at the repository root, and the workflow's in `cre-workflow/.env.example`. Copy the file and fill it in; nothing else needs configuring.